Neural Network – Formulas Summary

Liam Beaudet-Nallet

October 2026

In this document, "." corresponds to the matrix product while "x" corresponds to multiplication.

1 For 1 neuron

Definitions

Let m be the number of examples and 10 the number of features.

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_{10}^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_{10}^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \cdots & x_{10}^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times 10} \qquad W = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{10} \end{bmatrix} \in \mathbb{R}^{10 \times 1} \qquad b \in \mathbb{R}$$

Model

$$Z = X \cdot W + b$$
$$A = \frac{1}{1 + e^{-Z}}$$

Cost function

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} y \times \log(A) + (1 - y) \times \log(1 - A)$$

Gradient descent

$$W_{n+1} = W_n - \alpha \frac{\partial \mathcal{L}}{\partial W} \qquad \left(\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{m} X^T \cdot (A - y) \right)$$
$$b_{n+1} = b_n - \alpha \frac{\partial \mathcal{L}}{\partial b} \qquad \left(\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \sum (A - y) \right)$$

2 Neural Network

Notation

- $W_{ij}^{[l]}$: weight of layer l, for neuron i of that layer, connected to input j.
- $b_i^{[l]}$: bias of neuron i in layer l.

- $z_i^{[l]}$: linear output (pre-activation) of neuron i in layer l.
- $a_i^{[l]}$: activation (output) of neuron i in layer l.
- $Z^{[l]}$, $A^{[l]}$, $W^{[l]}$, $b^{[l]}$: matrices/vectors gathering these quantities for the whole layer l.

Two conventions for $Z^{[1]}$

The number m of training examples can be placed either on the *rows* or on the *columns* of the matrices. Both conventions are equivalent: one is obtained from the other by transposition. Starting from the row version $Z = X \cdot W + b$, we transpose the matrix product and the bias to get the transpose of Z:

$$(X \cdot W + b)^T = W^T \cdot X^T + b^T.$$

The weight matrix of the row version is therefore the transpose of the one in the column version, and b goes from a row vector to a column vector. The column version has the advantage of making the computation easier to visualize *row by row*: each row of $W^{[1]}$ holds the weights of one neuron, so each row of $Z^{[1]}$ is the output of one neuron for all examples, as if the data flowed through the neurons laid out in a column.

Convention 1: examples on the rows (shown here with 2 neurons)

$$Z^{[1]} = \underbrace{\begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} \end{bmatrix}}_{X \in \mathbb{R}^{m \times 2}} \cdot \underbrace{\begin{bmatrix} w_{11}^{[1]} & w_{21}^{[1]} \\ w_{12}^{[1]} & w_{22}^{[1]} \end{bmatrix}}_{W^{[1]} \in \mathbb{R}^{2 \times 2}} + \underbrace{\begin{bmatrix} b_1^{[1]} & b_2^{[1]} \end{bmatrix}}_{b^{[1]} \in \mathbb{R}^{1 \times 2}} = \underbrace{\begin{bmatrix} z_1^{[1](1)} & z_2^{[1](1)} \\ z_1^{[1](2)} & z_2^{[1](2)} \\ \vdots & \vdots \\ z_1^{[1](m)} & z_2^{[1](m)} \end{bmatrix}}_{Z^{[1]} \in \mathbb{R}^{m \times 2}}$$

Convention 2: examples on the columns (shown here with 3 neurons, and the one used in the rest of this section)

$$Z^{[1]} = \underbrace{\begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} \\ w_{31}^{[1]} & w_{32}^{[1]} \end{bmatrix}}_{W^{[1]} \in \mathbb{R}^{3 \times 2}} \cdot \underbrace{\begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(m)} \end{bmatrix}}_{X \in \mathbb{R}^{2 \times m}} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{bmatrix}}_{b^{[1]} \in \mathbb{R}^{3 \times 1}} = \underbrace{\begin{bmatrix} z_1^{[1](1)} & z_1^{[1](2)} & \cdots & z_1^{[1](m)} \\ z_2^{[1](1)} & z_2^{[1](2)} & \cdots & z_2^{[1](m)} \\ z_3^{[1](1)} & z_3^{[1](2)} & \cdots & z_3^{[1](m)} \end{bmatrix}}_{Z^{[1]} \in \mathbb{R}^{3 \times m}}$$

Forward propagation

$$\begin{aligned} Z^{[1]} &= W^{[1]} \cdot X + b^{[1]} \\ A^{[1]} &= \frac{1}{1 + e^{-Z^{[1]}}} \\ Z^{[2]} &= W^{[2]} \cdot A^{[1]} + b^{[2]} \\ A^{[2]} &= \frac{1}{1 + e^{-Z^{[2]}}} \end{aligned}$$

Cost function

$$\mathcal{L} = -\frac{1}{m} \sum y \times \log(A^{[2]}) + (1 - y) \times \log(1 - A^{[2]})$$

Gradients for backpropagation

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \times \frac{\partial A^{[2]}}{\partial Z^{[2]}} \times \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \times \frac{\partial A^{[2]}}{\partial Z^{[2]}} \times \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \times \frac{\partial A^{[2]}}{\partial Z^{[2]}} \times \frac{\partial Z^{[2]}}{\partial A^{[1]}} \times \frac{\partial A^{[1]}}{\partial Z^{[1]}} \times \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \times \frac{\partial A^{[2]}}{\partial Z^{[2]}} \times \frac{\partial Z^{[2]}}{\partial A^{[1]}} \times \frac{\partial A^{[1]}}{\partial Z^{[1]}} \times \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Simplified gradients

We set

$$dZ_2 = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \times \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$dZ_1 = dZ_2 \times \frac{\partial Z^{[2]}}{\partial A^{[1]}} \times \frac{\partial A^{[1]}}{\partial Z^{[1]}}$$

so that the gradients become

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = dZ_2 \times \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ_2 \times \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = dZ_1 \times \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = dZ_1 \times \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

That gives

$$dZ_2 = A^{[2]} - y$$

$$dZ_1 = W^{[2]T} \cdot dZ_2 \times A^{[1]} \left(1 - A^{[1]}\right)$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{1}{m}\, dZ_2 \cdot A^{[1]T}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{1}{m} \sum_{\text{axis 1}} dZ_2$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{1}{m}\, dZ_1 \cdot X^T$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{1}{m} \sum_{\text{axis 1}} dZ_1$$

3 Concepts

Broadcasting

Definition. Broadcasting is the mechanism that allows an operation (addition, multiplication, ...) between arrays of *different* shapes, by automatically "stretching" the smaller array so that both shapes match, without actually copying the data.

Rule. Compare the shapes of the two arrays starting from the *last* dimension and moving left. Two dimensions are compatible if:

- they are equal, or
- one of them is equal to 1. If so, then the dimension is stretched to match the other.

If a dimension is missing, it is treated as 1. If a pair of dimensions is neither equal nor 1, the operation fails.

Example. In $Z = X \cdot W + b$, the product $X \cdot W$ has shape $(m, 1)$ and b is a scalar: b is broadcast and added to each of the m rows.

Over-fitting

Definition. Overfitting occurs when a model fits the training data too closely (including its noise), so that the training error is very low but the error on new, unseen data is high: the model does not generalize.

Solution. Reduce the model's ability to memorize the training set:

- get more training data;
- add regularization (L2 penalty on the weights, dropout);
- use early stopping, monitoring the error on a validation set;
- simplify the model (fewer layers or neurons).